analytical_exercises/plots_em.py

```
import os
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # needed for 3D surface plots

# -------------------------------------------------
# 0. Ensure 'plots/' directory exists
# -------------------------------------------------
# If 'plots' does not exist, it is created. If it exists, nothing happens.
os.makedirs("plots", exist_ok=True)

# -------------------------------------------------
# 1. EM parameter values (SCALE parameterization)
# -------------------------------------------------
# Professor's model:
#   p(x1 | θ1) = (1/θ1) * exp(-x1 / θ1),  x1 ≥ 0
#   p(x2 | θ2) = (1/θ2),                  0 ≤ x2 ≤ θ2
#
# θ(0) = (θ1(0), θ2(0)) = (2, 3)
theta0_1 = 2.0   # θ1(0): scale (mean) before EM
theta0_2 = 3.0   # θ2(0): upper bound of uniform before EM

# θ(1) = (θ1(1), θ2(1)) = (7/3, 5)
theta1_1 = 7.0 / 3.0  # θ1(1): scale (mean) after EM
theta1_2 = 5.0        # θ2(1): upper bound of uniform after EM

# -------------------------------------------------
# 2. Define x1 and x2 ranges
# -------------------------------------------------
# x1 from 0 to 6 (covers samples 1, 2, 4 and the exponential tail)
x1 = np.linspace(0, 6, 200)

# x2 ranges:
#   - Before EM: [0, θ2(0)] = [0, 3]
#   - After EM : [0, θ2(1)] = [0, 5]
x2_before = np.linspace(0, theta0_2, 150)
x2_after  = np.linspace(0, theta1_2, 150)

# Create 2D grids (mesh) of x1 and x2 values for surface plots
X1_before, X2_before = np.meshgrid(x1, x2_before)
X1_after,  X2_after  = np.meshgrid(x1, x2_after)

# -------------------------------------------------
# 3. Define p(x1, x2) before and after EM
# -------------------------------------------------
# Because p(x1, x2) = p(x1) * p(x2), we compute marginals and multiply.

# ---------- Before EM: θ(0) = (2, 3) ----------
# p(x1 | θ1(0)) = (1 / θ1(0)) * exp(-x1 / θ1(0)), x1 ≥ 0
p1_before = (1.0 / theta0_1) * np.exp(-X1_before / theta0_1)

```

```
# p(x2 | θ2(0)) = 1 / θ2(0), 0 ≤ x2 ≤ θ2(0)
# (we already restrict x2_before to [0, θ2(0)])
p2_before = (1.0 / theta0_2) * np.ones_like(X2_before)

# Joint density (properly normalized on [0, ∞) × [0, θ2(0)])
p_before = p1_before * p2_before

# ---------- After EM: θ(1) = (7/3, 5) ----------
# p(x1 | θ1(1)) = (1 / θ1(1)) * exp(-x1 / θ1(1)), x1 ≥ 0
p1_after = (1.0 / theta1_1) * np.exp(-X1_after / theta1_1)

# p(x2 | θ2(1)) = 1 / θ2(1), 0 ≤ x2 ≤ θ2(1)
p2_after = (1.0 / theta1_2) * np.ones_like(X2_after)

# Joint density (properly normalized on [0, ∞) × [0, θ2(1)])
p_after = p1_after * p2_after

# --------------------------------------------------
# 4. Plot the two surfaces side by side
# --------------------------------------------------
fig = plt.figure(figsize=(10, 4))

# Before EM (θ(0))
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.plot_surface(X1_before, X2_before, p_before)
ax1.set_title('p(x1, x2) at θ(0) = (2, 3)')
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('density')

# After EM (θ(1))
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
ax2.plot_surface(X1_after, X2_after, p_after)
ax2.set_title('p(x1, x2) at θ(1) = (7/3, 5)')
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_zlabel('density')

plt.tight_layout()

# --------------------------------------------------
# 5. Save figure inside 'plots/' directory
# --------------------------------------------------
output_path = "plots/EM_Plots.png"
fig.savefig(output_path)

```